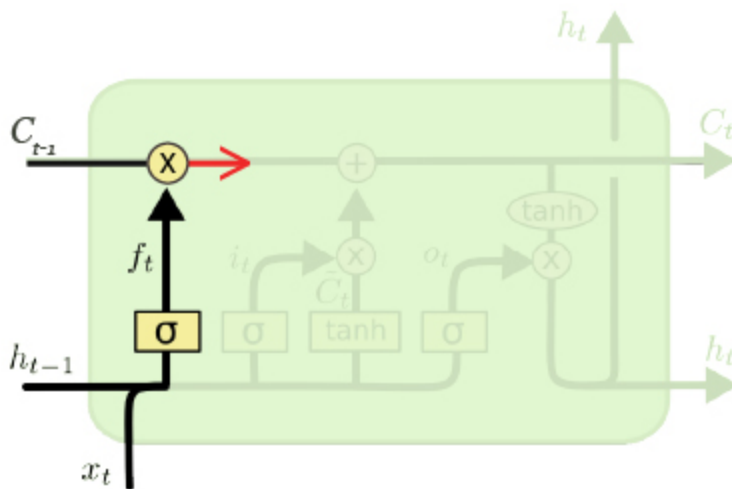


✓ The Forget Gate: High-Level Overview



🧻 Why do we need to remove information?

When an LSTM processes a sequence, as new information arrives (x_t), some older context stored in the Long-Term Memory (C_{t-1}) might lose its importance or relevance. If we keep everything, the memory will become cluttered. The Forget Gate's job is to inspect the memory and remove (forget) the information that is no longer needed.

📥 The Inputs

To make this decision, the Forget Gate looks at three things:

1. C_{t-1} : Previous Cell State (The Long-Term Memory).
2. h_{t-1} : Previous Hidden State (The Short-Term Memory).
3. x_t : Current Input (The current word/data).

⚙️ How it works (The Mechanism)

1. **Concatenation:** First, the short-term memory (h_{t-1}) and the current input (x_t) are combined or concatenated together: $[h_{t-1}, x_t]$.
2. **Neural Network (Sigmoid Layer):** This combined vector is passed through a neural network layer (with a Sigmoid activation function). The number of nodes in this network determines the shape of the output. This layer generates the **Forget Vector** (f_t).
3. **Pointwise Operation:** Finally, a pointwise multiplication ($\otimes$) happens between the old memory (C_{t-1}) and the forget vector (f_t). This operation strictly decides which

specific pieces of information to erase and which to keep.

🔴 ফরগেট গেট (Forget Gate): হাই-লেভেল ওভারভিউ - (বাংলা নোট)

🔧 আমাদের কেন তথ্য মুছে ফেলার প্রয়োজন হয়?

যখন একটি মডেল নতুন ডেটা (x_t) গ্রহণ করে, তখন লং-টার্ম মেমরিতে (C_{t-1}) থাকা আগের কিছু কনটেন্ট বা তথ্যের প্রয়োজনীয়তা কমে যেতে পারে। যদি আমরা সব তথ্যই রেখে দিই, তবে মেমরিতে জ্যাম লেগে যাবে। তাই Forget Gate-এর মূল কাজ হলো মেমরি চেক করা এবং অপ্রয়োজনীয় তথ্যগুলো মুছে ফেলা।

📦 ইনপুটসমূহ

কী মুছে ফেলতে হবে, সেই সিদ্ধান্ত নেওয়ার জন্য Forget Gate তিনটি জিনিসের দিকে লক্ষ্য রাখে: ১. C_{t-1} : আগের সেল স্টেট (লং-টার্ম মেমরি)। ২. h_{t-1} : আগের হিডেন স্টেট (শর্ট-টার্ম মেমরি)। ৩. x_t : বর্তমান ইনপুট (বর্তমান শব্দ বা ডেটা)।

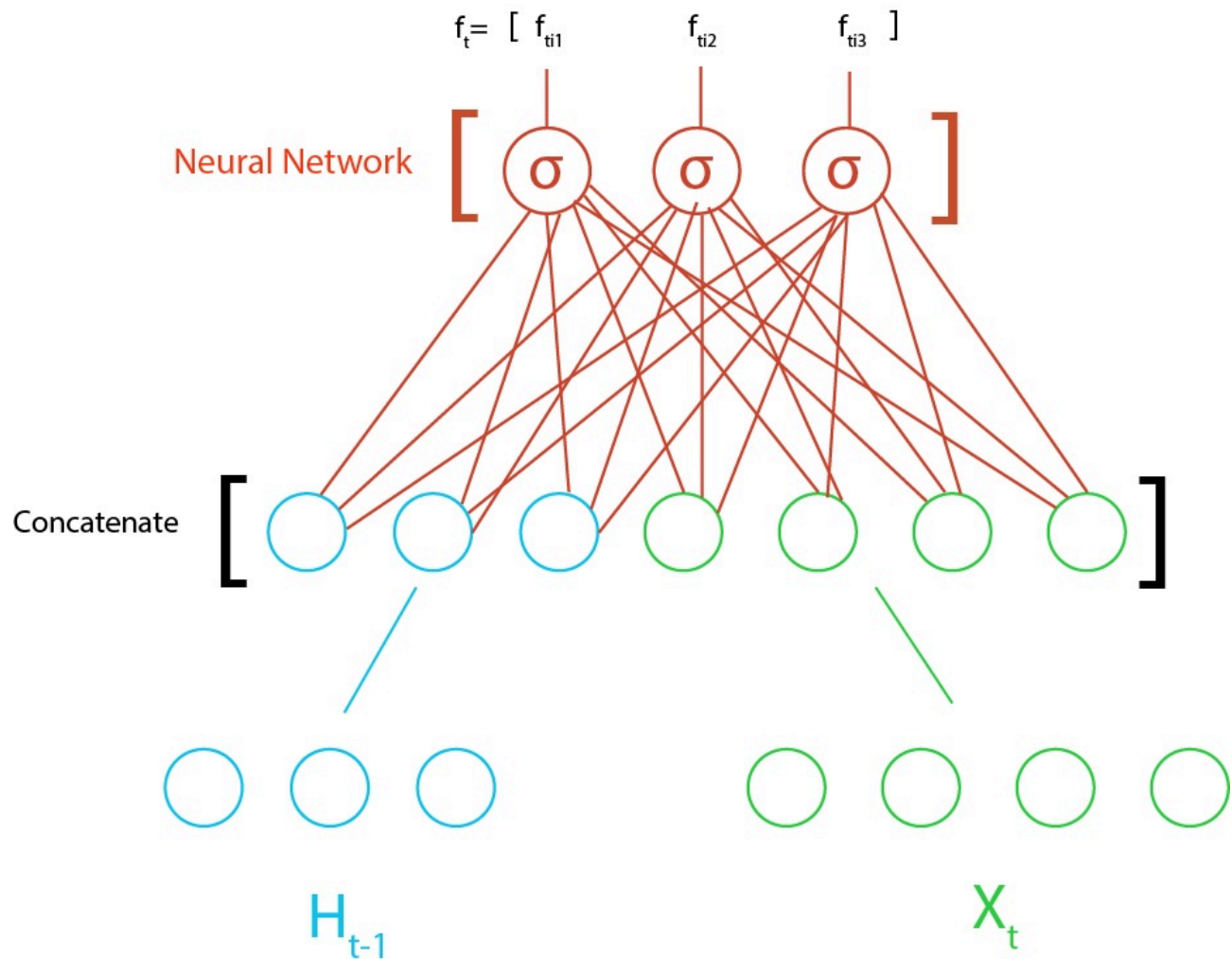
⚙️ এটি কীভাবে কাজ করে (কাজের ধাপ)

১. **Concatenation (যুক্ত হওয়া)**: একদম প্রথমেই শর্ট-টার্ম মেমরি (h_{t-1}) এবং বর্তমান ইনপুট (x_t) একসাথে জোড়া লেগে যায়: $[h_{t-1}, x_t]$ । ২. **Neural Network (সিগময়েড লেয়ার)**: এই কম্বাইন্ড ভেক্টরটি এরপর একটি নিউরাল নেটওয়ার্ক লেয়ারের (যাতে Sigmoid ফাংশন থাকে) ভেতর দিয়ে যায়। এই লেয়ারে কয়টি নোড আছে, তার ওপর ভিত্তি করে একটি নতুন আউটপুট তৈরি হয়, যাকে বলা হয় **Forget Vector (f_t)**। ৩. **Pointwise Operation (পয়েন্টওয়াইজ গুণ)**: সবশেষে, আগের মেমরি (C_{t-1}) এবং নতুন তৈরি হওয়া f_t -এর মধ্যে পয়েন্টওয়াইজ গুণ ($\otimes$) হয়। এই গুণের মাধ্যমেই চূড়ান্ত সিদ্ধান্ত নেওয়া হয় যে মেমরির কোন অংশটি মুছে যাবে আর কোনটি থেকে যাবে।

এখন আমাদের পরবর্তী টাগেট একদম সেট করা আছে! এরপর আমরা দেখব C_{t-1} , h_{t-1} , x_t এবং নিউরাল নেটওয়ার্কের শেপ বা ডাইমেনশনগুলো কেমন হয় এবং গাণিতিকভাবে f_t কীভাবে গঠিত হয়।



Forget Gate Calculation: Part 1 (Shape & Dimension Setup)



🎯 Our Core Objectives

To fully understand the Forget Gate, we need to mathematically achieve two things:

1. Calculate the Forget Vector (f_t).
2. Perform the pointwise multiplication: $C_{t-1} \otimes f_t$.

To do this, we must first confirm the shapes (dimensions) of all our variables. Since abstract formulas can be confusing, let's use a practical, realistic example based on your diagram.

📐 The Dimension Setup

Everything depends on the Neural Network layer's hyperparameter.

- **Hyperparameter:** Let's assume our neural network has **3 nodes (neurons)**.
- **The Golden Rule Consequence:** Because the hidden size is 3, the memory states must also be exactly size 3.

- C_{t-1} shape = 3
- H_{t-1} shape = 3
- **The Input (X_t):** Let's assume our input word, after tokenization and One-Hot Encoding, has a shape of 4.
- $X_t = [x_{i1}, x_{i2}, x_{i3}, x_{i4}]$ (Shape = 4)

⚙️ The Neural Network Parameters (Weights & Biases)

Now, H_{t-1} and X_t will concatenate and enter the neural network.

- **Concatenated Input:** Size 3 (H_{t-1}) + Size 4 (X_t) = **7 inputs**.
- **Weights (W):** Since 7 inputs are connecting to 3 neurons, the network will have $7 \times 3 = 21$ **distinct weights**.
- **Biases (b):** Each of the 3 neurons gets its own bias, so there are **3 biases**.



The Final Output (f_t)

Inside the 3 neurons, the calculations are wrapped in a **Sigmoid (σ)** activation function. Since there are 3 neurons, the final output vector will naturally have a shape of 3.

- $f_t = [f_{ti1}, f_{ti2}, f_{ti3}]$



ফরগেট গেট ক্যালকুলেশন: পার্ট ১ (শেপ এবং ডাইমেনশন) - বাংলা নোট



আমাদের মূল লক্ষ্য

Forget Gate-এর কাজ সম্পূর্ণ করার জন্য আমাদের গাণিতিকভাবে দুটি কাজ করতে হবে: ১. Forget Vector (f_t) ক্যালকুলেট করা। ২. C_{t-1} এবং f_t -এর মধ্যে পয়েন্টওয়াইজ গুণ ($C_{t-1} \otimes f_t$) করা। এই কাজগুলো করার জন্য আগে আমাদের সবগুলোর শেপ বা ডাইমেনশন কনফার্ম করে নিতে হবে। যেহেতু অ্যাবস্ট্রাক্ট সূত্র বোঝা কঠিন, তাই চলো একটি বাস্তব উদাহরণ তৈরি করি।



শেপ এবং ডাইমেনশন সেটআপ

পুরো হিসাবটি নির্ভর করে নিউরাল নেটওয়ার্কের নোড সংখ্যার ওপর।

- **হাইপারপ্যারামিটার:** ধরি, আমাদের নিউরাল নেটওয়ার্কে ৩টি নোড (নিউরন) আছে।
- **সূত্র অনুযায়ী:** যেহেতু নোড ৩টি, তাই আমাদের মেমরি স্টেটগুলোর সাইজও ঠিক ৩ হতে হবে।
- C_{t-1} -এর সাইজ = ৩
- H_{t-1} -এর সাইজ = ৩

- **ইনপুট (X_t):** ধরি, টোকেনাইজেশন এবং One-Hot এনকোডিংয়ের পর আমাদের বর্তমান ইনপুটের সাইজ হয়েছে ৪।
- $X_t = [x_{i1}, x_{i2}, x_{i3}, x_{i4}]$ (সাইজ = ৪)

⚙️ নিউরাল নেটওয়ার্কের প্যারামিটার (Weights ও Biases)

এখন H_{t-1} এবং X_t জোড়া (Concatenate) লেগে নিউরাল নেটওয়ার্কে প্রবেশ করবে।

- **Concatenated ইনপুট:** সাইজ ৩ (H_{t-1}) + সাইজ ৪ (X_t) = মোট ৭টি ইনপুট।
- **ওয়েট (Weights, W):** যেহেতু ৭টি ইনপুট ৩টি নিউরনের সাথে যুক্ত হচ্ছে, তাই এখানে ওয়েট থাকবে $৭ \times ৩ = ২১$ টি।
- **বায়াস (Biases, b):** ৩টি নিউরনের প্রতিটির জন্য ১টি করে মোট ৩টি বায়াস থাকবে।



চূড়ান্ত আউটপুট (f_t)

এই ৩টি নিউরনের ভেতরে ডেটাগুলো হিসাব হয়ে **Sigmoid (σ)** অ্যাক্টিভেশন ফাংশনের ভেতর দিয়ে যাবে। যেহেতু নিউরন ৩টি, তাই চূড়ান্ত আউটপুট ভেক্টরের সাইজও হবে ঠিক ৩।

- $f_t = [f_{ti1}, f_{ti2}, f_{ti3}]$



Forget Gate Calculation: Part 2 (Forming the Equation)

Since the Forget Gate uses a Neural Network layer, it must follow the standard linear equation: $y = Wx + b$, followed by an activation function.



Formulating the Equation

- **The Input (x):** Our input is the concatenated vector of the previous hidden state and the current input. So, $x = [h_{t-1}, x_t]$.
- **The Weights (W):** We call the weight matrix for the forget gate W_f .
- **The Bias (b):** We call the bias vector for the forget gate b_f .
- **The Activation:** We pass the entire linear result through a Sigmoid function (σ).

Putting it all together, the formal equation for the Forget Gate is:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$



Proving the Output Shape (Matrix Multiplication)

Let's use the dimensions from our previous example to prove why f_t results in a shape of 3. In linear algebra, a matrix dot product ($A \cdot B$) is only possible if the inner dimensions

match, and the result takes the outer dimensions.

- Dimension of $W_f = 3 \times 7$ (3 nodes, 7 concatenated inputs)
- Dimension of $[h_{t-1}, x_t] = 7 \times 1$ (Column vector of 7 inputs)

The Dot Product:

$$(3 \times 7) \cdot (7 \times 1) \rightarrow \text{Resulting Dimension} = 3 \times 1$$

Since the resulting matrix is 3×1 , adding the bias (b_f , which is also 3×1) and applying the Sigmoid function keeps the final shape at 3×1 . Thus, f_t perfectly outputs a vector of **3 values**!



ফরগেট গেট ক্যালকুলেশন: পার্ট ২ (সমীকরণ গঠন) - বাংলা নোট

যেহেতু Forget Gate-এ একটি নিউরাল নেটওয়ার্ক লেয়ার ব্যবহার করা হয়, তাই এটিকে অবশ্যই স্ট্যান্ডার্ড লিনিয়ার ইকুয়েশন $y = Wx + b$ মেনে চলতে হবে এবং এর সাথে একটি অ্যাক্টিভেশন ফাংশন যুক্ত করতে হবে।



সমীকরণ বা ইকুয়েশন তৈরি

- **ইনপুট (x):** আমাদের ইনপুট হলো আগের হিডেন স্টেট এবং বর্তমান ইনপুটের জোড়া লাগানো (Concatenated) ভেক্টর। অর্থাৎ, $x = [h_{t-1}, x_t]$ ।
- **ওয়েট (W):** Forget Gate-এর ওয়েট ম্যাট্রিক্সকে আমরা ধরি W_f ।
- **বায়াস (b):** Forget Gate-এর বায়াস ভেক্টরকে আমরা ধরি b_f ।
- **অ্যাক্টিভেশন:** লিনিয়ার ইকুয়েশনের পুরো রেজাল্টটিকে আমরা Sigmoid ফাংশনের (σ) ভেতর দিয়ে পাঠাই।

সবগুলো মান একসাথে বসালে Forget Gate-এর গাণিতিক সমীকরণটি দাঁড়ায়:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$



আউটপুট শেপ প্রমাণ (ম্যাট্রিক্স মাল্টিপ্লিকেশন)

চলো আমাদের আগের উদাহরণের ডাইমেনশনগুলো ব্যবহার করে প্রমাণ করি কেন f_t -এর সাইজ ৩ হবে। লিনিয়ার অ্যালজেব্রার নিয়ম অনুযায়ী, ম্যাট্রিক্স গুণের ($A \cdot B$) ক্ষেত্রে ভেতরের ডাইমেনশন সমান হতে হয় এবং ফলাফল বাইরের ডাইমেনশনের সাইজ অনুযায়ী হয়।

- W_f -এর ডাইমেনশন = 3×7 (৩টি নোড, ৭টি ইনপুট)
- $[h_{t-1}, x_t]$ -এর ডাইমেনশন = 7×1 (৭টি ইনপুটের কলাম ভেক্টর)

ম্যাট্রিক্স গুণ (Dot Product):

$$(3 \times 7) \cdot (7 \times 1) \rightarrow \text{ফলাফলের ডাইমেনশন} = 3 \times 1$$

যেহেতু গুণফলটি একটি 3×1 সাইজের ম্যাট্রিক্স, তাই এর সাথে বায়াস (b_f , যা নিজেও 3×1) যোগ করলে এবং Sigmoid অ্যাপ্লাই করলেও ফাইনাল সাইজ 3×1 -ই থাকে। এভাবেই f_t নিখুঁতভাবে ৩টি ভ্যালুর একটি আউটপুট তৈরি করে!



Forget Gate Calculation: Part 3 (Applying the Forget Vector)

Our final task for the Forget Gate is to apply the generated forget vector (f_t) to the old memory (C_{t-1}) using a **pointwise multiplication** ($\otimes$).

Depending on the values inside f_t (which range from 0 to 1 due to the Sigmoid function), there are **3 possible cases**:

● Case 1: Remember Everything (100% Retained)

If the neural network decides all past information is highly relevant, the Sigmoid outputs 1s.

- C_{t-1} : $[1.2, 3.1, 2.1]$
- f_t : $[1, 1, 1]$
- **Pointwise Calculation:** $[(1.2 \times 1), (3.1 \times 1), (2.1 \times 1)]$
- **Result:** $C_{t-1} \otimes f_t = [1.2, 3.1, 2.1]$
- *Logic:* The old memory passes through exactly as it was. Nothing is forgotten.

● Case 2: Forget Completely (0% Retained)

If the neural network decides the old context is completely useless now, the Sigmoid outputs 0s.

- C_{t-1} : $[1.2, 3.1, 2.1]$
- f_t : $[0, 0, 0]$
- **Pointwise Calculation:** $[(1.2 \times 0), (3.1 \times 0), (2.1 \times 0)]$
- **Result:** $C_{t-1} \otimes f_t = [0, 0, 0]$
- *Logic:* The memory is completely wiped clean.

● Case 3: Partial Forgetting / Keeping

In reality, the model usually keeps some parts, reduces the importance of others, and completely forgets a few.

- C_{t-1} : $[1.2, 3.1, 2.1]$

- $f_t: [1, 0.2, 0]$
- **Pointwise Calculation:** $[(1.2 \times 1), (3.1 \times 0.2), (2.1 \times 0)]$
- **Result:** $C_{t-1} \otimes f_t = [1.2, 0.62, 0]$
- *Logic:* The first feature is fully kept, the second feature is reduced to 20% of its original strength, and the third feature is completely deleted!



ফরগেট গেট ক্যালকুলেশন: পার্ট ৩ (ভেক্টর অ্যাপ্লাই করা)

- বাংলা নোট

Forget Gate-এর একদম শেষের কাজ হলো তৈরি হওয়া ফরগেট ভেক্টর (f_t)-কে আগের মেমরির (C_{t-1}) সাথে পয়েন্টওয়াইজ গুণ ($\otimes$) করা।

যেহেতু Sigmoid ফাংশনের কারণে f_t -এর মান সবসময় ০ থেকে ১-এর মধ্যে থাকে, তাই এখানে মূলত ৩টি ঘটনা (Cases) ঘটতে পারে:

● কেইস ১: সম্পূর্ণ মনে রাখা (Remember Everything)

যদি মডেল মনে করে আগের সব তথ্যই বর্তমানের জন্য খুব গুরুত্বপূর্ণ, তবে Sigmoid আউটপুট হিসেবে ১ দেয়।

- $C_{t-1}: [1.2, 3.1, 2.1]$
- $f_t: [1, 1, 1]$
- পয়েন্টওয়াইজ ক্যালকুলেশন: $[(1.2 \times 1), (3.1 \times 1), (2.1 \times 1)]$
- ফলাফল: $C_{t-1} \otimes f_t = [1.2, 3.1, 2.1]$
- *লজিক:* আগের মেমরির পুরো হুবহু কপি পরের স্টেপে চলে যাবে, অর্থাৎ সেল স্টেট থেকে কিছুই মুছে যায়নি।

● কেইস ২: পুরোপুরি ভুলে যাওয়া (Forget Completely)

যদি মডেল মনে করে আগের তথ্যের আর কোনোই দরকার নেই, তবে Sigmoid আউটপুট হিসেবে ০ দেয়।

- $C_{t-1}: [1.2, 3.1, 2.1]$
- $f_t: [0, 0, 0]$
- পয়েন্টওয়াইজ ক্যালকুলেশন: $[(1.2 \times 0), (3.1 \times 0), (2.1 \times 0)]$
- ফলাফল: $C_{t-1} \otimes f_t = [0, 0, 0]$
- *লজিক:* আগের মেমরির সব তথ্য পুরোপুরি মুছে বা রিসেট হয়ে জিরো হয়ে গেছে।

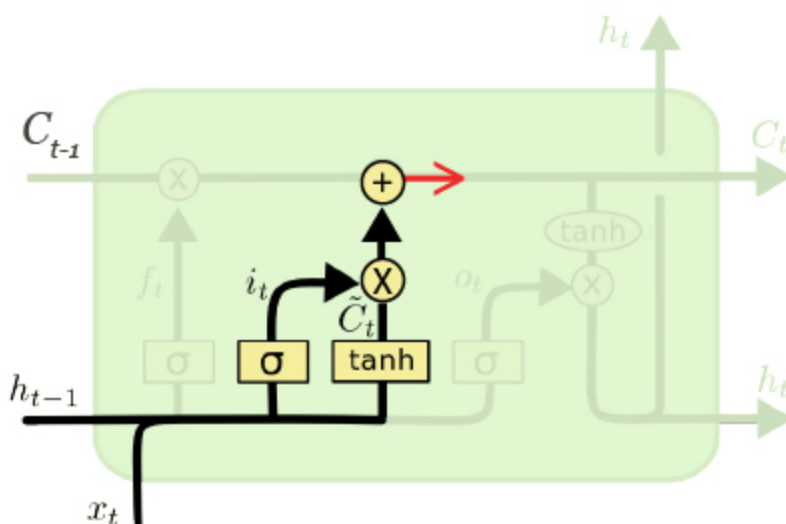
● কেইস ৩: আংশিক মনে রাখা বা ভুলে যাওয়া (Partial Memory)

বাস্তবে মডেল সাধারণত কিছু তথ্য পুরোপুরি রেখে দেয়, কিছু তথ্যের গুরুত্ব কমিয়ে দেয় এবং কিছু তথ্য পুরোপুরি মুছে ফেলে।

- C_{t-1} : [1.2, 3.1, 2.1]
- f_t : [1, 0.2, 0]
- পয়েন্টওয়াইজ ক্যালকুলেশন: $[(1.2 \times 1), (3.1 \times 0.2), (2.1 \times 0)]$
- ফলাফল: $C_{t-1} \otimes f_t = [1.2, 0.62, 0]$
- লজিক: প্রথম ফিচারটি ১০০% মনে রেখেছে, দ্বিতীয় ফিচারটির গুরুত্ব কমিয়ে ২০%-এ নিয়ে এসেছে (৩.১ থেকে ০.৬২), এবং তৃতীয় ফিচারটি পুরোপুরি ডিলিট করে দিয়েছে!

💡 চূড়ান্ত কথা: এখান থেকেই পরিষ্কার বোঝা যায় যে, f_t -এর মান ০ থেকে ১-এর মধ্যে থাকার কারণেই মডেল নিখুঁতভাবে সিদ্ধান্ত নিতে পারে সে কোন তথ্য কতটুকু রাখবে বা কতটুকু মুছে ফেলবে।

✖ 📦 The Input Gate: Part 1 (High-Level Overview)



🎯 What is the Input Gate?

While the Forget Gate's job is to remove irrelevant information, the **Input Gate's job is to add new, relevant information** to the Cell State (Long-Term Memory).

Why do we need to store new info? Because as the sequence progresses (e.g., reading the next word in a sentence), new context arrives. The network uses the previous short-term memory (h_{t-1}) and the current input (x_t) to decide what new context is worth remembering.

🧠 The Components and the "Job Interview" Analogy

Storing new information happens in a multi-step process involving two main vectors:

1. $\tilde{C}_t$ (**Candidate Cell State**): * The Analogy: Think of this as the **Job Candidates**. When a company is hiring, people give interviews. They *might* get added to the

team, but it is not 100% guaranteed yet.

- *In LSTM*: This vector holds all the potential new information that *could* be added to the cell state.
2. i_t (**Input Vector**): * *The Analogy*: Think of this as the **HR Manager or Interviewer**.
- *In LSTM*: It acts as a filter. It decides exactly which parts of the Candidate Cell State ($\tilde{C}_t$) are actually useful and should be allowed to enter the Long-Term Memory.

The 3 Steps of Calculation

To complete the Input Gate mechanism and update our memory, we need to calculate three things:

1. **Calculate $\tilde{C}_t$** : Generate the pool of potential new information.
2. **Calculate i_t** : Generate the filter to evaluate the candidates.
3. **Calculate the New Cell State (C_t)**: Combine the remaining old memory (what we kept from the Forget Gate: $C_{t-1} \otimes f_t$) with the newly filtered candidate memory.

ইনপুট গেট (Input Gate): পার্ট ১ (হাই-লেভেল ওভারভিউ) - বাংলা নোট

ইনপুট গেটের কাজ কী?

Forget Gate-এর কাজ ছিল অপ্রয়োজনীয় তথ্য মুছে ফেলা, আর **Input Gate**-এর কাজ হলো সেল স্টেটে (লং-টার্ম মেমরিতে) নতুন প্রয়োজনীয় তথ্য যুক্ত করা।

তথ্য কেন স্টোর করতে হবে? কারণ বাক্য বা সিকোয়েন্স যত এগোতে থাকে, নতুন নতুন কনটেন্ট বা অর্থ তৈরি হয়। ইনপুট গেট আগের শর্ট-টার্ম মেমরি (h_{t-1}) এবং বর্তমান ইনপুট (x_t)-এর ওপর ভিত্তি করে নির্ধারণ করে যে, নতুন কোন তথ্যগুলো মনে রাখা উচিত।

এর উপাদান এবং "ইন্টারভিউ ক্যান্ডিডেট" লজিক

নতুন তথ্য স্টোর করার এই প্রক্রিয়াটি মূলত দুটি ভেক্টরের মাধ্যমে কয়েকটি ধাপে কাজ করে:

১. $\tilde{C}_t$ (**Candidate Cell State**): * *লজিক*: তুমি যেমন বলেছ, একে ইন্টারভিউ ক্যান্ডিডেটদের সাথে তুলনা করা যায়। যারা ইন্টারভিউ দিচ্ছে, তারা টিমে জয়েন করতে পারে, তবে ১০০% গ্যারান্টি নেই।

- *LSTM-এ*: এই ভেক্টরটি মূলত সেই সম্ভাব্য নতুন তথ্যগুলোকে ধারণ করে, যেগুলো সেল স্টেটে যুক্ত হওয়ার যোগ্যতা রাখে।

২. i_t (**Input Vector**): * *লজিক*: এটি মূলত ইন্টারভিউয়ার বা ফিল্টার (**Filter**) হিসেবে কাজ করে।

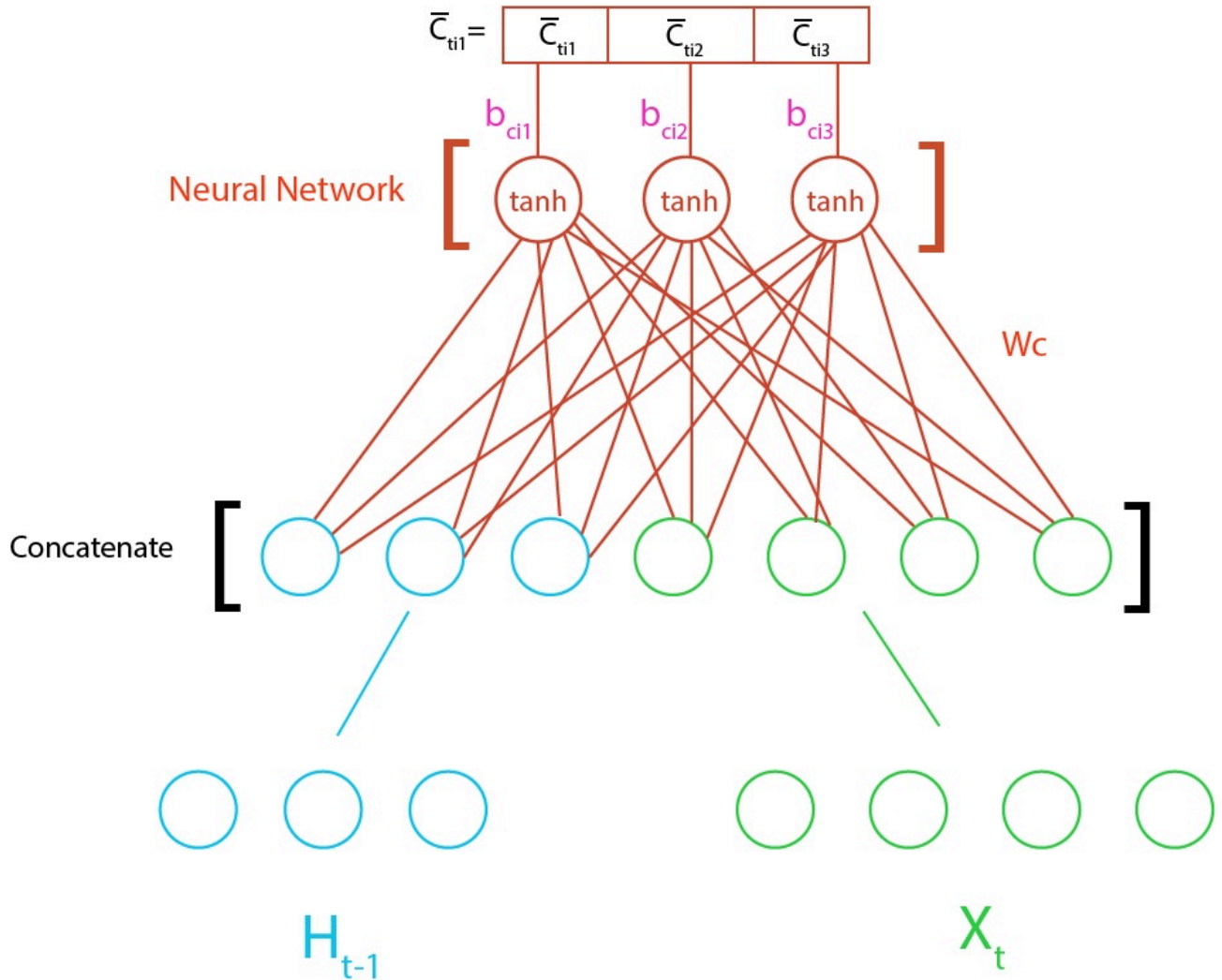
- *LSTM-এ*: ক্যান্ডিডেট সেল স্টেটের ($\tilde{C}_t$) কোন অংশটুকু আসলেই কাজের এবং কোন অংশটুকু বাদ দেওয়া উচিত, সেটি i_t নির্ধারণ করে।

📋 ক্যালকুলেশনের ৩টি ধাপ

পুরো মেকানিজমটি সম্পন্ন করার জন্য আমাদের ৩টি জিনিস ক্যালকুলেট করতে হবে: ১. $\tilde{C}_t$ ক্যালকুলেট করা: সম্ভাব্য নতুন তথ্যের (ক্যান্ডিডেটদের) তালিকা তৈরি করা। ২. i_t ক্যালকুলেট করা: ফিল্টারটি তৈরি করা, যা ক্যান্ডিডেটদের যাচাই করবে। ৩. নতুন সেল স্টেট (C_t) ক্যালকুলেট করা: Forget Gate পার হয়ে আসা আগের মেমরির ($C_{t-1} \otimes f_t$) সাথে ফিল্টার করা নতুন তথ্যগুলো যোগ করে বর্তমান মেমরি আপডেট করা।



Input Gate Part 2: Calculating Candidate Cell State ($\tilde{C}_t$)



Now we will calculate the **Candidate Cell State** ($\tilde{C}_t$). This vector holds the new information that *might* be added to the memory. Since we already know the dimensions from the Forget Gate, this calculation is super easy!



Recalling the Dimensions

- H_{t-1} shape = 3
- X_t shape = 4
- Neural Network Nodes = 3



Formulating the Equation

Just like any standard neural network layer, we use the linear equation $y = Wx + b$ and pass it through an activation function. Here, the activation function is $\tanh$ (which outputs values between -1 and 1).

- **Input (x):** The concatenated vector $[h_{t-1}, x_t]$.
- **Weights (W):** The weight matrix for the candidate state is called W_c .
- **Bias (b):** The bias vector for the candidate state is called b_c .

Putting it together, the formal equation is:

$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$



Matrix Multiplication & Output Shape

Let's verify the dimensions to ensure the math works:

- Dimension of $W_c = 3 \times 7$ (3 nodes, 7 concatenated inputs)
- Dimension of $[h_{t-1}, x_t] = 7 \times 1$ (Column vector of 7 inputs)

The Dot Product:

$$(3 \times 7) \cdot (7 \times 1) \rightarrow \text{Resulting Dimension} = 3 \times 1$$

After adding the bias (b_c , which is 3×1) and applying the $\tanh$ function, the final output $\tilde{C}_t$ is perfectly a vector of shape 3.



ইনপুট গেট পার্ট ২: ক্যান্ডিডেট সেল স্টেট ($\tilde{C}_t$)

ক্যালকুলেশন - বাংলা নোট

এবার আমরা **Candidate Cell State** ($\tilde{C}_t$) ক্যালকুলেট করব। এই ভেক্টরটি মূলত সেই সম্ভাব্য নতুন তথ্যগুলোকে ধারণ করে, যা মেমরিতে যুক্ত হতে পারে। যেহেতু আমরা Forget Gate থেকে

ডাইমেনশনের হিসাবগুলো আগেই জেনে গেছি, তাই এই ক্যালকুলেশনটা একদম পানির মতো সহজ!



ডাইমেনশন রিক্যাপ

- H_{t-1} -এর সাইজ = ৩
- X_t -এর সাইজ = ৪
- নিউরাল নেটওয়ার্ক নোড = ৩



সমীকরণ বা ইকুয়েশন তৈরি

যেকোনো সাধারণ নিউরাল নেটওয়ার্ক লেয়ারের মতোই এখানে আমরা $y = Wx + b$ ইকুয়েশন ব্যবহার করব এবং সেটিকে একটি অ্যাক্টিভেশন ফাংশনের ভেতর দিয়ে পাঠাব। ইনপুট গেটের এই অংশে অ্যাক্টিভেশন ফাংশন হিসেবে $\tanh$ ব্যবহার করা হয় (যা -১ থেকে ১ এর মধ্যে ভ্যালু দেয়)।

- ইনপুট (x): জোড়া লাগানো ভেক্টর $[h_{t-1}, x_t]$ ।
- ওয়েট (W): ক্যান্ডিডেট স্টেটের জন্য ওয়েট ম্যাট্রিক্সকে আমরা ধরি W_c ।
- বায়াস (b): ক্যান্ডিডেট স্টেটের বায়াস ভেক্টরকে আমরা ধরি b_c ।

সবগুলো মান একসাথে বসালে গাণিতিক সমীকরণটি দাঁড়ায়:

$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$



ম্যাট্রিক্স মাল্টিপ্লিকেশন এবং আউটপুট শেপ

চলো ডাইমেনশনগুলো চেক করে দেখি গাণিতিক হিসাবটি ঠিক আছে কি না:

- W_c -এর ডাইমেনশন = 3×7 (৩টি নোড, ৭টি ইনপুট)
- $[h_{t-1}, x_t]$ -এর ডাইমেনশন = 7×1 (৭টি ইনপুটের কলাম ভেক্টর)

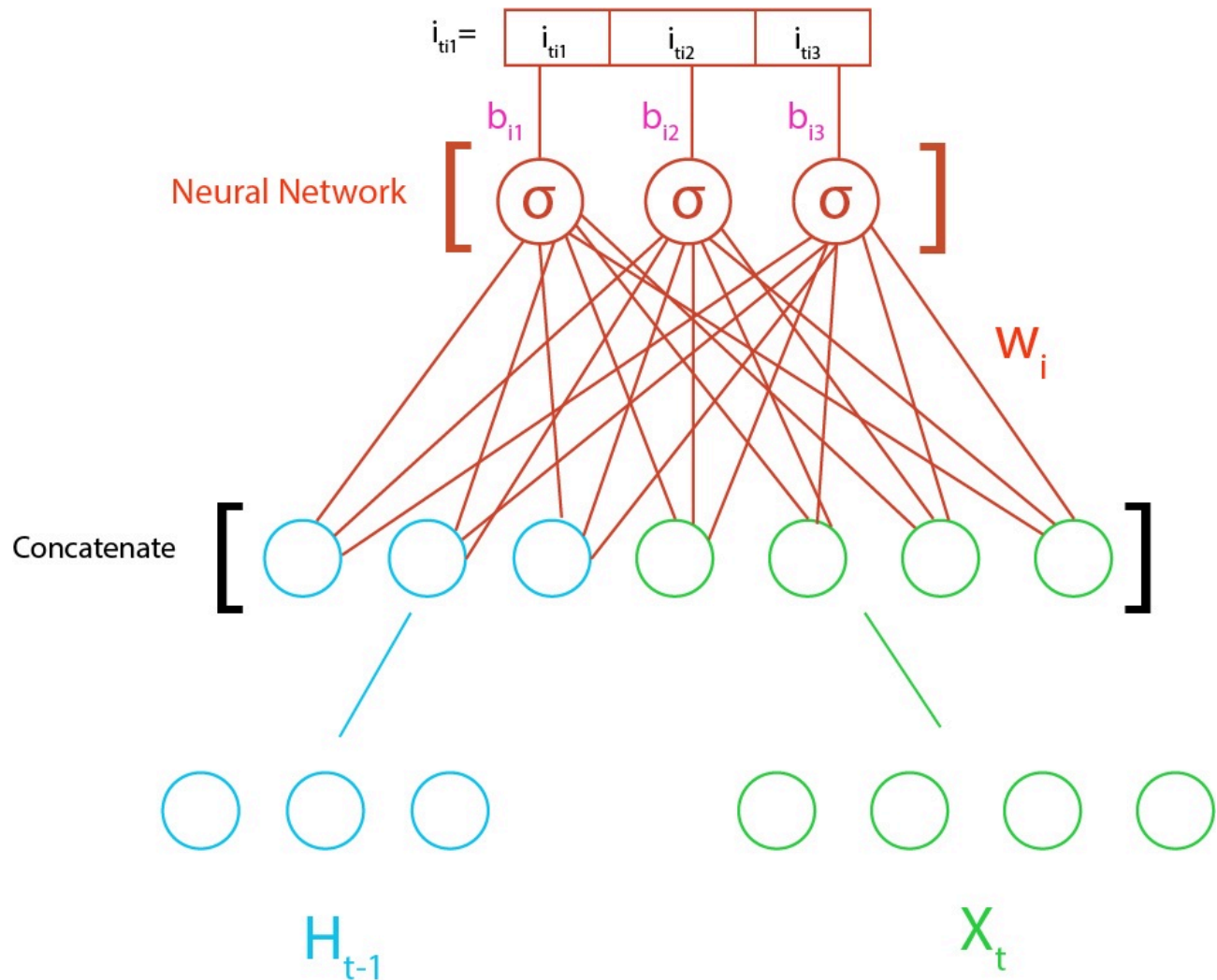
ম্যাট্রিক্স গুণ (Dot Product):

$$(3 \times 7) \cdot (7 \times 1) \rightarrow \text{ফলাফলের ডাইমেনশন} = 3 \times 1$$

যেহেতু গুণফলটি একটি 3×1 সাইজের ম্যাট্রিক্স, এর সাথে বায়াস (b_c) যোগ করার পর $\tanh$ অ্যাপ্লাই করলেও ফাইনাল আউটপুট $\tilde{C}_t$ -এর সাইজ নিখুঁতভাবে ৩-ই থাকে।



Input Gate Part 3: Calculating the Input Vector (i_t)



🎯 The Role of i_t

Now we need to calculate the **Input Vector** (i_t). If $\tilde{C}_t$ represents the "job candidates," then i_t is the "interviewer." Its job is to act as a filter to decide which specific parts of the Candidate Cell State ($\tilde{C}_t$) are actually worthy of entering the main Cell State.

📐 Recalling the Dimensions

Because we already know our network size, the dimensions are straightforward:

- H_{t-1} shape = 3, X_t shape = 4 $\rightarrow$ Concatenated shape = 7×1
- Neural Network Nodes = 3

📝 Formulating the Equation

Since i_t is a filter that decides "how much" information to let through (from 0% to 100%), we must use the **Sigmoid** (σ) activation function, not $\tanh$.

- **Weights (W):** We call the weight matrix for this layer W_i .

- **Bias (b):** We call the bias vector for this layer b_i .

Putting it all into our standard neural network equation:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$



Matrix Multiplication & Output Shape

Let's look at the math to ensure it works perfectly:

- Dimension of $W_i = 3 \times 7$
- Dimension of $[h_{t-1}, x_t] = 7 \times 1$

The Dot Product:

$$(3 \times 7) \cdot (7 \times 1) \rightarrow \text{Resulting Dimension} = 3 \times 1$$

After matrix multiplication, adding the bias (b_i , shape 3×1), and applying the Sigmoid function, the final output i_t is perfectly a vector of shape **3**.



Conclusion: We now have our filter ready! We are completely prepared to filter the candidate information.



ইনপুট গেট পার্ট ৩: ইনপুট ভেক্টর (i_t) ক্যালকুলেশন - বাংলা নোট



i_t -এর কাজ কী?

এবার আমরা **Input Vector (i_t)** ক্যালকুলেট করব। $\tilde{C}_t$ যদি "ইন্টারভিউ ক্যান্ডিডেট" হয়, তবে i_t হলো "ইন্টারভিউয়ার"। এর কাজ হলো একটি ফিল্টার হিসেবে কাজ করা এবং নির্ধারণ করা যে, ক্যান্ডিডেট সেল স্টেটের ($\tilde{C}_t$) কোন অংশটুকু আসলেই মূল সেল স্টেটে যাওয়ার যোগ্যতা রাখে।



ডাইমেনশন রিক্যাপ

যেহেতু আমাদের আগের ডাইমেনশনগুলো মনে আছে, তাই হিসাবটা একদম পানির মতো:

- H_{t-1} -এর সাইজ ৩ এবং X_t -এর সাইজ ৪ $\rightarrow$ জোড়া লাগানোর পর সাইজ = 7×1
- নিউরাল নেটওয়ার্ক নোড = ৩



সমীকরণ বা ইকুয়েশন তৈরি

যেহেতু i_t একটি ফিল্টার হিসেবে কাজ করে এবং সিদ্ধান্ত নেয় "কতটুকু" ডেটা যাবে (০% নাকি ১০০%), তাই এখানে $\tanh$ -এর বদলে অবশ্যই **Sigmoid (σ)** অ্যাক্টিভেশন ফাংশন ব্যবহার করতে হবে।

- ওয়েট (W): এই লেয়ারের ওয়েট ম্যাট্রিক্সকে আমরা ধরি W_i ।
- বায়াস (b): এই লেয়ারের বায়াস ভেক্টরকে আমরা ধরি b_i ।

সবগুলো মান লিনিয়ার ইকুয়েশনে বসালে সমীকরণটি দাঁড়ায়:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

 ম্যাট্রিক্স মাল্টিপ্লিকেশন এবং আউটপুট শেপ

চলো ডাইমেনশনের গুণটা মিলিয়ে দেখি:

- W_i -এর ডাইমেনশন = 3×7
- $[h_{t-1}, x_t]$ -এর ডাইমেনশন = 7×1

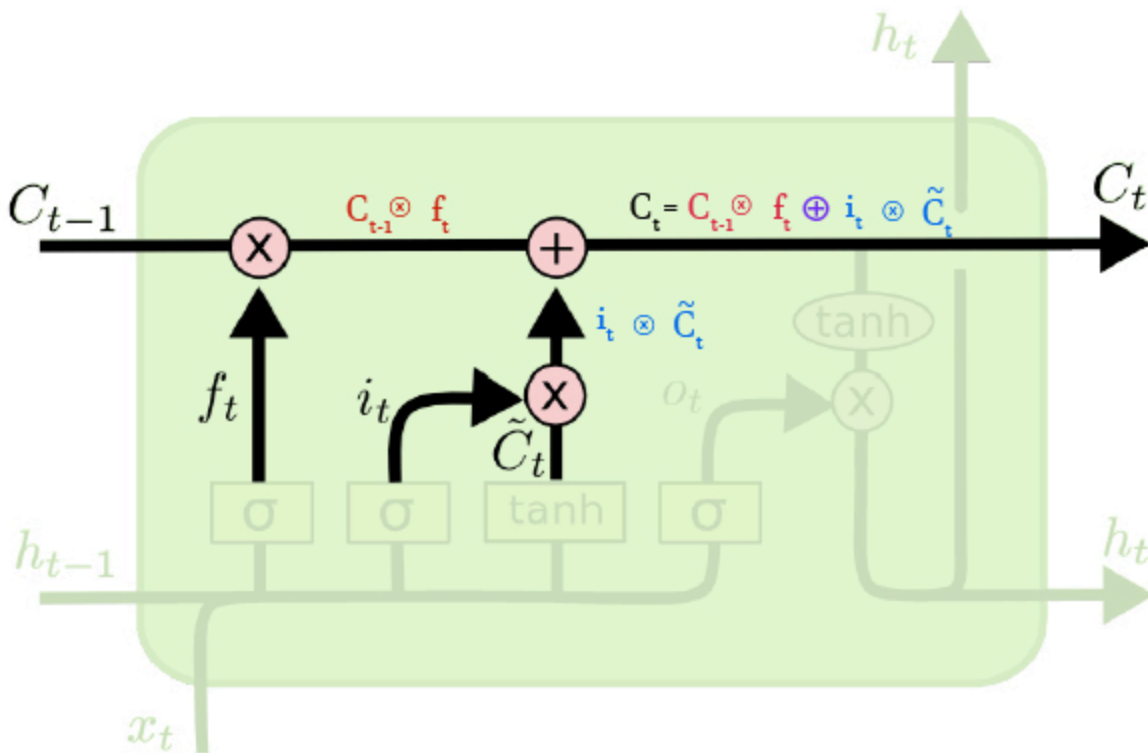
ম্যাট্রিক্স গুণ (Dot Product):

$$(3 \times 7) \cdot (7 \times 1) \rightarrow \text{ফলাফলের ডাইমেনশন} = 3 \times 1$$

এই গুণের পর বায়াস (b_i) যোগ করে Sigmoid অ্যাপ্লাই করলে ফাইনাল আউটপুট i_t -এর সাইজ নিখুঁতভাবে ৩-ই থাকে।

 উপসংহার: আমাদের ফিল্টার এখন তৈরি! ক্যান্ডিডেট ডেটাগুলোকে ফিল্টার করার জন্য আমরা এখন সম্পূর্ণ প্রস্তুত।

Input Gate Part 4: The Final Memory Update (C_t)



We are now at the most critical junction of the LSTM cell. We have our candidate information ($\tilde{C}_t$) and our filter (i_t). Now, we need to apply the filter and officially update the Cell State (Long-Term Memory) to C_t .

How the Filtering Works ($i_t \otimes \tilde{C}_t$)

We perform a pointwise multiplication between the filter (i_t) and the candidates ($\tilde{C}_t$). Since i_t contains values from 0 to 1 (thanks to Sigmoid), 3 cases can occur:

1. **Reject Everything (Filter out all):**

- $\tilde{C}_t = [1, 0.5, 1]$ and $i_t = [0, 0, 0]$
- $i_t \otimes \tilde{C}_t = [(1 \times 0), (0.5 \times 0), (1 \times 0)] = [0, 0, 0]$

2. **Accept Everything (Keep all):**

- $\tilde{C}_t = [1, 0.5, 1]$ and $i_t = [1, 1, 1]$
- $i_t \otimes \tilde{C}_t = [(1 \times 1), (0.5 \times 1), (1 \times 1)] = [1, 0.5, 1]$

3. **Partial Filtering (Keep some, reduce some, drop some):**

- $\tilde{C}_t = [1, 0.5, 1]$ and $i_t = [1, 0.5, 0]$
- $i_t \otimes \tilde{C}_t = [(1 \times 1), (0.5 \times 0.5), (1 \times 0)] = [1, 0.25, 0]$

The Master Equation: Calculating C_t

Now we combine the past and the present.

- **From Forget Gate:** The old memory we decided to keep: ($f_t \otimes C_{t-1}$)
- **From Input Gate:** The new filtered memory we want to add: ($i_t \otimes \tilde{C}_t$)

We combine them using pointwise addition ($\oplus$):

$$C_t = (f_t \otimes C_{t-1}) \oplus (i_t \otimes \tilde{C}_t)$$

Tracking the Information Flow (Example)

Let's track a real data flow based on this equation:

- **Assume Old Memory:** $C_{t-1} = [4, 5, 6]$
- **Forget Decision:** The model decides to keep everything $\rightarrow f_t = [1, 1, 1]$.
- **Result:** $f_t \otimes C_{t-1} = [4, 5, 6]$
- **Input Decision:** The model decides NO new information is useful right now $\rightarrow i_t \otimes \tilde{C}_t = [0, 0, 0]$.

- **Final Output (C_t):**
- $C_t = [4, 5, 6] \oplus [0, 0, 0] = [4, 5, 6]$ (The long-term memory moves to the next time step exactly as it was!)

ইনপুট গেট পার্ট ৪: চূড়ান্ত মেমরি আপডেট (C_t) - বাংলা নোট

আমরা এখন LSTM সেলের সবচেয়ে গুরুত্বপূর্ণ অংশে আছি। আমাদের কাছে সম্ভাব্য নতুন তথ্যের ক্যান্ডিডেট ($\tilde{C}_t$) এবং ফিল্টার (i_t) দুটোই আছে। এখন আমাদের কাজ হলো ফিল্টারটি প্রয়োগ করা এবং মূল সেল স্টেট বা লং-টার্ম মেমরিকে আপডেট করে C_t তৈরি করা।

ফিল্টারিং কীভাবে কাজ করে ($i_t \otimes \tilde{C}_t$)

আমরা ফিল্টার (i_t) এবং ক্যান্ডিডেটদের ($\tilde{C}_t$) মধ্যে পয়েন্টওয়াইজ গুণ করি। যেহেতু Sigmoid-এর কারণে i_t -এর মান ০ থেকে ১ এর মধ্যে থাকে, তাই এখানে ৩ ধরনের ঘটনা ঘটতে পারে:

১. সব বাদ দিয়ে দেওয়া (Reject All):

- $\tilde{C}_t = [1, 0.5, 1]$ এবং $i_t = [0, 0, 0]$
- $i_t \otimes \tilde{C}_t = [(1 \times 0), (0.5 \times 0), (1 \times 0)] = [0, 0, 0]$

২. সব রেখে দেওয়া (Accept All):

- $\tilde{C}_t = [1, 0.5, 1]$ এবং $i_t = [1, 1, 1]$
- $i_t \otimes \tilde{C}_t = [(1 \times 1), (0.5 \times 1), (1 \times 1)] = [1, 0.5, 1]$

৩. আংশিক ফিল্টারিং (কিছু কমানো, কিছু বাদ দেওয়া, কিছু রাখা):

- $\tilde{C}_t = [1, 0.5, 1]$ এবং $i_t = [1, 0.5, 0]$
- $i_t \otimes \tilde{C}_t = [(1 \times 1), (0.5 \times 0.5), (1 \times 0)] = [1, 0.25, 0]$

মাস্টার ইকুয়েশন: C_t ক্যালকুলেশন

এবার আমরা অতীত এবং বর্তমানকে এক করব।

- **Forget Gate** থেকে প্রাপ্ত: পুরোনো মেমরির যে অংশটুকু আমরা রেখে দিয়েছি: ($f_t \otimes C_{t-1}$)
- **Input Gate** থেকে প্রাপ্ত: নতুন ফিল্টার করা মেমরি যা আমরা যুক্ত করতে চাই: ($i_t \otimes \tilde{C}_t$)

এই দুটি অংশকে আমরা পয়েন্টওয়াইজ যোগের ($\oplus$) মাধ্যমে যুক্ত করি:

$$C_t = (f_t \otimes C_{t-1}) \oplus (i_t \otimes \tilde{C}_t)$$

ডেটা ফ্লো ট্র্যাক করা (উদাহরণ)

চলো এই সমীকরণের মাধ্যমে ডেটা কীভাবে ফ্লো করে তা দেখি:

- ধরি পুরোনো মেমরি: $C_{t-1} = [4, 5, 6]$
- **Forget সিদ্ধান্ত:** মডেল সিদ্ধান্ত নিলো পুরোনো সব তথ্যই রেখে দেবে $\rightarrow f_t = [1, 1, 1]$
- ফলাফল: $f_t \otimes C_{t-1} = [4, 5, 6]$
- **Input সিদ্ধান্ত:** মডেল সিদ্ধান্ত নিলো এই মুহুর্তে নতুন কোনো তথ্য যোগ করার দরকার নেই $\rightarrow i_t \otimes \tilde{C}_t = [0, 0, 0]$
- চূড়ান্ত আউটপুট (C_t):
- $C_t = [4, 5, 6] \oplus [0, 0, 0] = [4, 5, 6]$ (লং-টার্ম মেমরিটি কোনো পরিবর্তন ছাড়াই হুবহু পরবর্তী টাইম স্টেপের জন্য পাস হয়ে গেল!)



Input Gate Part 5: The "Why" Behind LSTM (Achieving Long-Term Memory)

We have looked at the calculations, but **why** do we set up the math this way? Your realization about the data flow perfectly answers the biggest question in deep learning sequence models.



The RNN Problem vs. The LSTM Solution

As we discussed earlier, Simple RNNs only have Short-Term Memory (STM). They suffer from the Vanishing Gradient problem, meaning long-term context gets diluted and eventually lost as time steps increase.

LSTM was designed specifically to solve this. Through the mathematical operations of the Forget Gate and Input Gate, LSTM successfully achieves **Long-Term Memory (LTM)**.



The Power of Gate Control (The Example)

Let's look at the critical example you provided:

- $C_{t-1} = [4, 5, 6]$
- If the context is highly important, the Forget Gate outputs $f_t = [1, 1, 1]$.
- If no new information is needed, the Input Gate outputs an update of $[0, 0, 0]$.
- **Result:** $C_t = [4, 5, 6]$

Why is this magical? Because the exact original vector $[4, 5, 6]$ traveled to the next time step without any loss, dilution, or alteration. By simply outputting 1s (to keep) and

Os (to block), the network has total control over carrying long-term context for as many time steps as it wants!



What is an LSTM, really?

At its absolute core, an LSTM network is simply a highly controlled "**Vector Transfer System**".

- Information in the real world (like text or words) is converted into numbers (embeddings/vectors).
- These vectors enter the LSTM.
- The gates simply act as traffic lights, calculating whether to transfer, block, or modify these vectors based on the context required.



ইনপুট গেট পার্ট ৫: LSTM-এর আসল জাদুকরী ক্ষমতা (Long-Term Memory অর্জন) - বাংলা নোট

আমরা এতক্ষণ ধরে যে ক্যালকুলেশনগুলো দেখলাম, সেগুলো কেন এভাবেই সাজানো হয়েছে? ডেটা ফ্লো নিয়ে তোমার এই রিয়েলাইজেশনটি ডিপ লার্নিংয়ের সিকোয়েন্স মডেলের সবচেয়ে বড় প্রশ্নের উত্তর দেয়।



Simple RNN-এর সমস্যা বনাম LSTM-এর সমাধান

আমরা একদম শুরুতেই পড়েছিলাম যে, Simple RNN-এ শুধুমাত্র Short-Term Memory (STM) থাকে। এটি Vanishing Gradient সমস্যায় ভোগে, যার ফলে সময় বা স্টেপ বাড়ার সাথে সাথে এর লং-টার্ম কনটেক্সট বা পুরনো মেমরি মুছে যেতে থাকে।

LSTM তৈরিই করা হয়েছে এই সমস্যাটি সমাধান করার জন্য। Forget Gate এবং Input Gate-এর এই গাণিতিক অপারেশনগুলোর মাধ্যমেই LSTM সফলভাবে **Long-Term Memory (LTM)** অর্জন করতে পারে।



গেট কন্ট্রলের আসল ক্ষমতা (তোমার উদাহরণ)

তোমার দেওয়া সেই চমৎকার উদাহরণটি খেয়াল করি:

- $C_{t-1} = [4, 5, 6]$
- যদি এই তথ্যটি খুব জরুরি হয়, তবে Forget Gate আউটপুট দেবে $f_t = [1, 1, 1]$ ।
- যদি নতুন কোনো তথ্যের প্রয়োজন না হয়, তবে Input Gate আউটপুট দেবে $[0, 0, 0]$ ।
- ফলাফল: $C_t = [4, 5, 6]$

এটি কেন এত গুরুত্বপূর্ণ? কারণ একদম শুরুর আসল ভেক্টরটি $[4, 5, 6]$ কোনো ধরনের পরিবর্তন বা ডেটা লস ছাড়াই সফলভাবে পরবর্তী টাইম স্টেপে পার হয়ে গেল। শুধু ১ (রাখার জন্য) এবং ০ (বাদ

দেওয়ার জন্য) আউটপুট দেওয়ার মাধ্যমে এই নেটওয়ার্কটি তার লং-টার্ম কনটেক্সটকে ঠিক ততটাই ক্যারি করতে পারে, যতটা তার প্রয়োজন!



দিনশেষে LSTM আসলে কী?

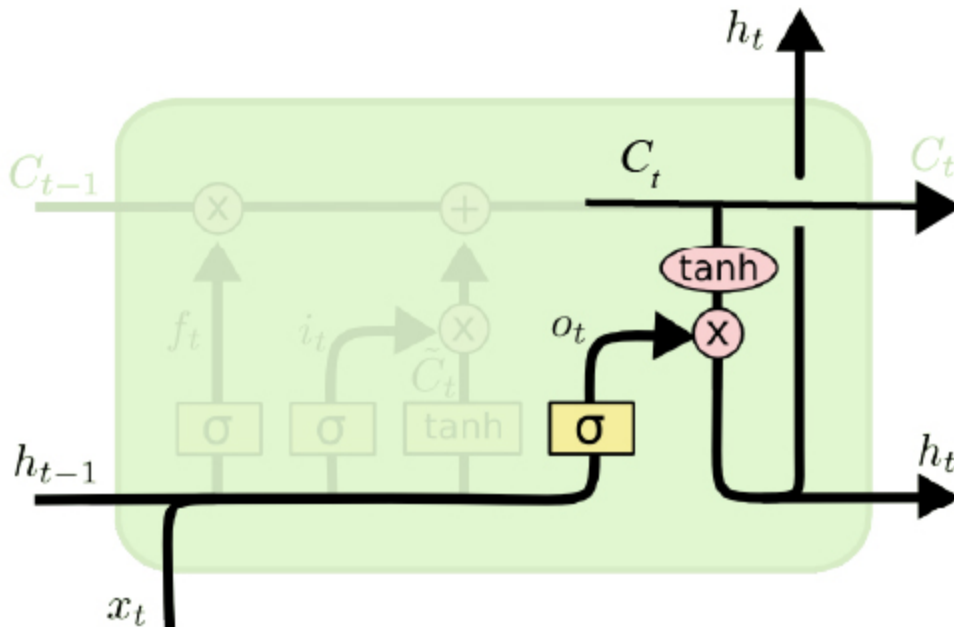
সবচেয়ে সহজ ভাষায় বলতে গেলে, একটি LSTM নেটওয়ার্ক হলো অত্যন্ত নিয়ন্ত্রিত একটি "ভেক্টর ট্রান্সফার সিস্টেম (Vector Transfer System)"।

- বাস্তব জগতের ডেটাকে (যেমন: টেক্সট বা শব্দ) প্রথমে নাম্বারে বা ভেক্টরে কনভার্ট করা হয়।
- সেই ভেক্টরগুলো ডেটা হিসেবে মেমরিতে প্রবেশ করে।
- গেটগুলো মূলত ট্রাফিক সিগন্যালের মতো কাজ করে এবং গাণিতিক ক্যালকুলেশনের মাধ্যমে সিদ্ধান্ত নেয় যে— এই ভেক্টরগুলোকে কি সামনে ট্রান্সফার করা হবে, মুছে ফেলা হবে, নাকি মডিফাই করা হবে।



The Output Gate: Calculating the Hidden State

(h_t)



We have reached the final gate! The sole purpose of the Output Gate is to generate the new **Hidden State** (h_t), which acts as the Short-Term Memory and the actual output of the LSTM cell for the current time step.



Clearing a Common Misconception

Many people mistakenly think the Output Gate calculates *both* the Cell State (C_t) and the Hidden State (h_t). **This is false.** As we saw earlier, C_t (Long-Term Memory) is already calculated by combining the Forget and Input gates. The Output Gate simply *uses* the newly updated C_t to figure out what the new h_t should be.

⚙️ The 2-Step Calculation Process

To find h_t , the Output Gate creates a filter and applies it to a scaled version of the Cell State.

Step 1: Calculate the Output Filter (o_t) Just like the previous gates, we use the previous hidden state and current input to create a filter using the Sigmoid (σ) function.

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

Step 2: Scale C_t and Apply the Filter

- First, we pass the newly updated Cell State (C_t) through a $\tanh$ function. As you brilliantly explained with your example ($[4, 5, 6]$), this pushes all values to be strictly between -1 and 1: $\tanh(C_t)$.
- Then, we perform a pointwise multiplication ($\otimes$) between our filter (o_t) and the scaled cell state to get our final output.

$$h_t = o_t \otimes \tanh(C_t)$$

📖 আউটপুট গেট (Output Gate): হিডেন স্টেট (h_t)

ক্যালকুলেশন - বাংলা নোট

আমরা একদম শেষ গেটে পৌঁছে গেছি! আউটপুট গেটের একমাত্র কাজ হলো নতুন **Hidden State** (h_t) তৈরি করা। এই h_t -ই হলো মডেলের শর্ট-টার্ম মেমরি এবং বর্তমান টাইম স্টেপের জন্য মডেলের চূড়ান্ত আউটপুট।

💡 একটি সাধারণ ভুল ধারণার অবসান

অনেকেই মনে করে আউটপুট গেট হয়তো Cell State (C_t) এবং Hidden State (h_t) দুটোই আউটপুট দেয়। এটি সম্পূর্ণ ভুল। আমরা আগেই দেখেছি, Forget এবং Input গেটের মাধ্যমেই C_t (লং-টার্ম মেমরি) ক্যালকুলেট করা হয়ে গেছে। আউটপুট গেট মূলত সেই নতুন আপডেট হওয়া C_t -কে ব্যবহার করে নির্ধারণ করে যে, নতুন h_t কেমন হবে।

⚙️ ক্যালকুলেশনের ২টি ধাপ

h_t বের করার জন্য আউটপুট গেট একটি ফিল্টার তৈরি করে এবং সেটিকে C_t -এর ওপর প্রয়োগ করে।

ধাপ ১: আউটপুট ফিল্টার (o_t) তৈরি করা আগের গেটগুলোর মতোই, আগের হিডেন স্টেট এবং বর্তমান ইনপুটকে কাজে লাগিয়ে Sigmoid (σ) ফাংশনের মাধ্যমে একটি ফিল্টার তৈরি করা হয়।

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

ধাপ ২: C_t -কে স্কেল করা এবং ফিল্টার প্রয়োগ করা